

Dual-Arm Manipulation for Tic-Tac-Toe: Workspace Analysis and Collision-Aware Coordination

Shucheng JIA, Alec Henderson, Zhilin HUANG, Junyan ZHOU, Xingrui CHEN

ABSTRACT- This paper presents a dual-arm robotic system that autonomously plays Tic-Tac-Toe using RGB-D perception, game reasoning, and coordinated manipulation. This project first analyzes per-cell reachability using a MATLAB kinematic model, then introduces a workspace-partitioning strategy that assigns board regions to each arm and serializes motion in shared areas. Experiments conducted in Gazebo and on real Kinova Gen3 Lite robots demonstrate that this approach enables reliable, collision-free full-game execution. These results show that structured workspace allocation is an effective and practical method for safe dual-arm coordination.

I. INTRODUCTION

Dual-arm robotic manipulation enables efficient and flexible interaction in tabletop environments, but coordinating multiple manipulators in a shared workspace remains a nontrivial challenge [1][2]. Effective coordination requires the arms to behave predictably and avoid collision within an overlapping workspace. Without explicit mechanisms for cooperation, standard motion planners may produce inconsistent behavior or fail when both arms attempt to access similar areas of the workspace [3].

To study these challenges in a controlled and interpretable setting, this paper constructs a fully autonomous dual-arm system that plays Tic-Tac-Toe. Although the game logic is simple, the manipulation task reproduces core issues present in larger multi-arm systems: all nine cells of the 3×3 board lie inside the overlapping workspaces of both manipulators, meaning that either arm is physically capable of placing a piece in any location. This full redundancy allows both manipulators to effectively carry out their given task, but introduces the potential for collision in the shared workspace, especially when operating in independent planning environments. Such challenges present the need for a more structured coordination mechanism, which we seek to explore in this paper.

Our work focuses on analyzing the dual-arm workspace and developing a reliable method for collision-aware coordination. We begin with a systematic reachability study to determine which board cells are accessible to each arm and to identify regions where workspace contention is unavoidable[4]. Based on this analysis, this paper introduces a workspace-partition strategy that assigns specific board regions to each manipulator and employs a simple arbitration rule for shared cells such as the center. This approach reduces inter-arm interference and yields more predictable planning outcomes compared to naïve shared-workspace operation.

The complete system integrates RGB-D perception, rule-based game reasoning, and coordinated motion planning using MoveIt in both Gazebo simulation and a real dual-Kinova Gen3 Lite hardware setup. Our experiments—including reachability evaluation, inter-arm distance measurements, and repeated full-game trials—demonstrate that structured workspace allocation greatly improves safety and planning reliability. These findings provide practical guidance for designing robust multi-arm coordination strategies in broader collaborative robotic settings.

II. RELATED WORK

Cooperative dual-arm manipulation has been widely studied in recent years, with a strong emphasis on coordination, redundancy resolution, and collision avoidance in shared workspaces. Survey papers highlight that dual-arm systems must coordinate both in task space and joint space to avoid self-collisions and singularities while maintaining feasible motions for both arms[7]. Many works therefore frame multi-arm planning as a constrained optimization problem, using centralized inverse kinematics or hierarchical controllers to generate collision-free trajectories under geometric and kinematic constraints[6].

Another line of work reduces planning complexity by partitioning the workspace into zones and scheduling which robot is allowed to occupy each zone at a given time. In these methods, zones are treated as shared resources and non-overlap constraints are enforced to guarantee safety and avoid deadlock[5][8]. Such ideas have been applied mainly to industrial settings such as multi-robot assembly and material handling, often using relatively heavy optimization or constraint-programming machinery.

At the software level, motion-planning frameworks such as MoveIt provide widely used building blocks for kinematics, collision checking, and trajectory generation[3], and official Kinova ROS 2 drivers make it possible to deploy these planners on Gen3 and Gen3 Lite hardware. Our work builds on these tools but focuses on a lightweight, static workspace-partition strategy for a dual-arm Tic-Tac-Toe task, showing that even a simple geometric allocation scheme can improve safety and predictability in a compact shared workspace.

III. SYSTEM DESIGN & METHODS

Our system adopts a modular Perception–Planning–Execution architecture for controlling a dual-arm robotic platform in both Gazebo simulation and a real dual–Kinova Gen3 Lite setup. This architecture cleanly separates sensing, game-state reasoning, and low-level motion generation, enabling each subsystem to be developed, validated, and debugged in isolation. Such decoupling is essential for dual-arm operation, as it allows perception to remain stable across environments, planning to incorporate coordination constraints, and execution to guarantee safe motion without cross-module interference.

A. Perception–Planning–Execution Overview

An overhead RGB-D camera observes the board and pieces, publishing depth and color data into ROS 2. The perception node converts this stream into a filtered point cloud in the world frame, clusters individual blocks, and writes their 3D centroids into a structured world-state file. The planning layer reads this file, reconstructs the Tic-Tac-Toe board state, selects the next move and the responsible arm, and emits a high-level manipulation script. Finally, the execution layer parses this script and uses MoveIt within the appropriate ROS namespace to generate and execute collision-free trajectories for the left or right arm. The overall Perception–Planning–Execution pipeline and information flow between the two arms are summarized in Figure 1.

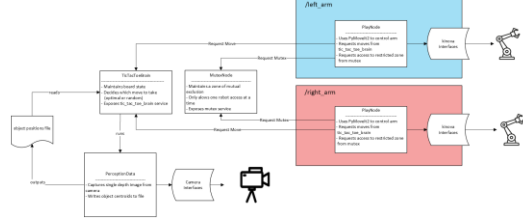


Figure 1 System diagram

B. Perception Pipeline

The perception subsystem is shared between simulation and hardware. In both cases, an overhead RGB-D sensor is mounted above the workspace. The raw point cloud, initially expressed in the camera frame, is transformed into a common world frame using TF2 so that estimated block positions are directly usable for motion planning.

Before clustering, the depth data undergoes a lightweight filtering stage to improve robustness across environments. We apply a pass-through filter to restrict the cloud to the board volume, remove statistical outliers caused by sensor noise, and suppress the table plane. This produces a clean, compact point cloud that is stable across both Gazebo and real hardware.

In addition to geometric filtering, the cloud is spatially partitioned with respect to the board’s centerline. Because all points are already expressed in the world frame, simple geometric relations can be evaluated directly at this stage. Points lying on

the left side of the board are labeled as belonging to the left workspace, and points on the right side as belonging to the right workspace. This early-stage region labeling simplifies downstream coordination logic by providing an explicit Left/Right association for each detected cluster.

On the filtered and region-labeled cloud, we apply a DBSCAN-based clustering algorithm to segment the scene into individual blocks. For each cluster, we compute a centroid and record its world-frame position, region label, and a unique identifier in the detected-objects file. This representation is intentionally simple: it captures only the geometric and spatial information required by the game logic and motion planner, while allowing the perception front end to be upgraded in the future without altering downstream components. By standardizing the TF tree and message interfaces, the same perception pipeline is used in both Gazebo and on the physical robots, enabling direct sim-to-real transfer without code modification.

The Figure 2 MoveIt/RViz view shows the dual arms anchored in a shared world frame, along with the board and game pieces represented as collision objects. The segmented blocks appear as clean, isolated clusters in the point cloud, demonstrating that filtering and DBSCAN clustering produce stable, planner-ready geometric data in simulation—the same structure later replicated on hardware.

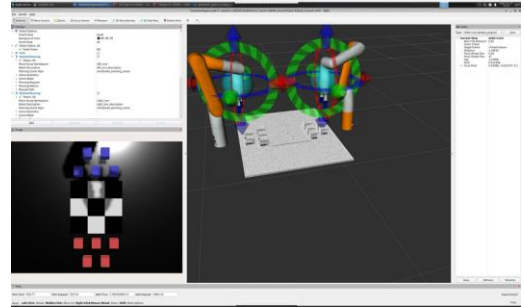


Figure 2 Dual-arm system loaded inside MoveIt with live RGB and point-cloud data

C. Game State Reconstruction and Action Script Generation

Given the YAML world state, the planning node projects the block centroids into the board frame and assigns each block to one of the nine cells by checking which nominal cell center it is closest to. With this mapping, the node reconstructs the full Tic-Tac-Toe board and checks whether the game has reached a terminal state. A rule-based decision engine then selects the robot’s next move according to an optimal strategy..

Once the target cell is chosen, the planner will generate a sequence of high-level actions describing the pick and place positions of the game piece. We also optionally serialize all motions to a YAML file for the purpose of replaying a particular sequence of moves if we notice errant behavior.

D. Dual-Arm ROS Architecture and Workspace Partition

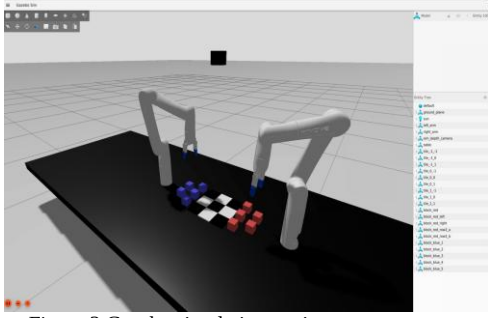


Figure 3 Gazebo simulation environment

To control two Kinova Gen3 Lite arms simultaneously, we adopt a distributed ROS 2 architecture in which each manipulator operates inside its own namespace (`/left_arm` and `/right_arm`). Within each namespace, the `ros2_kortex` driver, the MoveIt configuration, and all controllers are launched independently. Because the standard driver does not provide native multi-robot support, we developed custom launch scripts that inject namespaced XML parameters into each robot’s URDF at runtime. This ensures that TF frames, controller managers, and action servers remain fully isolated, allowing the high-level planner to command both arms uniformly while preventing cross-communication.

Although MoveIt can avoid collisions reactively, granting both manipulators unrestricted access to the board produced frequent interference due to the fully overlapping workspace. To address this, we introduce a structured workspace-partition strategy in which the board is divided into two lateral regions, each assigned to a specific arm, with only the center column treated as shared space. These assignments align with the left/right spatial labels generated by the perception pipeline, enabling the planner to reuse the geometric partition inferred from sensor data.

To safely manage access to the shared region, we implement a dedicated `MutexNode` that enforces strict mutual exclusion: only one arm may enter the shared workspace at a time, preventing contention and physical deadlocks. Combined with staggered transport heights for vertical separation, this geometry-informed coordination scheme produces safe, predictable dual-arm behavior even in a fully overlapping task space.

IV. SYSTEMATIC TEST PLAN

A. Objectives and Validation Strategy

The goal of our evaluation is not to test game logic, but to systematically characterize the dual-arm workspace and assess whether our coordination strategy is justified and safe before exercising the real hardware. Concretely, we aim to:

(i) determine per-cell reachability for both manipulators over the 3×3 board,

(ii) quantify how deeply the board lies inside each arm’s safe workspace, and

(iii) verify that the Tic-Tac-Toe task is fundamentally limited by coordination rather than by kinematic feasibility.

Because the physical Gen3 Lite manipulators are costly to operate and not designed to tolerate repeated collision events, we adopt a staged validation strategy. We first conduct systematic workspace and clearance analyses in MATLAB using a kinematic model of the dual-arm setup, allowing per-cell reachability and safety margins to be evaluated without physical risk. Only task configurations that satisfy these kinematic criteria are then tested in Gazebo, where the same Perception–Planning–Execution pipeline runs with full collision checking and dynamic simulation.

Finally, once both MATLAB and Gazebo indicate safe and feasible behavior, we perform a controlled set of real-world trials on the physical robots. This staged procedure minimizes the likelihood of hardware damage while still providing quantitative workspace verification and qualitative confirmation of system behavior on the actual platform.

B. MATLAB Workspace Model

To support systematic testing, we construct a simplified dual-arm workspace model in MATLAB. Each Gen3 Lite manipulator is modeled as a rigid-body kinematic chain with joint limits matching the physical hardware. The two bases are positioned in world coordinates to approximate the actual table-top arrangement used in our experiments. A planar board frame is defined at the table surface, where the nine Tic-Tac-Toe cells are represented as fixed target poses arranged in a 3×3 grid at a nominal grasp height.

For each arm, we use an inverse kinematics solver to determine if a collision-free joint configuration exists for a given cell pose. A cell is marked as reachable if the solver identifies at least one valid solution. To evaluate redundancy, we record how many valid solutions are found in total. We also approximate a safety margin by measuring the minimum distance from each target pose to the boundary of the other robot’s workspace. A clearance threshold of 0.15 m is used, and any configuration with a smaller margin is marked as potentially unsafe. The resulting model allows us to quantify per-cell reachability, redundancy, and clearance margins, establishing the baseline for the results presented in the following subsections

C. Test 1 – Per-Cell Reachability and Safety Margins

For each of the nine board cells, we define a nominal grasp pose at the cell center and query inverse kinematics for both manipulators. A cell is marked reachable for an arm if at least one joint configuration within limits is found. In addition, we approximate a safety margin by measuring the

minimum distance from the cell target to the opposite robot's workspace boundary and comparing it to a 0.15 m clearance threshold.

Cell	Reach (L)	Margin (L)	Reach (R)	Margin (R)	Status
1	1	0.52 m	1	0.37 m	FULL REDUNDANCY
2	1	0.45 m	1	0.45 m	FULL REDUNDANCY
3	1	0.37 m	1	0.52 m	FULL REDUNDANCY
4	1	0.52 m	1	0.36 m	FULL REDUNDANCY
5	1	0.44 m	1	0.44 m	FULL REDUNDANCY
6	1	0.36 m	1	0.52 m	FULL REDUNDANCY
7	1	0.52 m	1	0.37 m	FULL REDUNDANCY
8	1	0.45 m	1	0.45 m	FULL REDUNDANCY
9	1	0.37 m	1	0.52 m	FULL REDUNDANCY

Table 1. Per-cell reachability and clearance margins relative to a 0.15 m safety threshold.

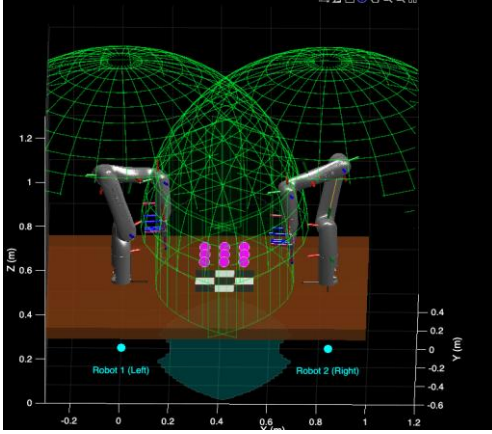


Figure 4 Dual-arm workspace envelopes in MATLAB with board placement.

The dual-arm workspace envelopes and board placement used in these computations are shown in Fig. 4. Numerical results are summarized in Table I: all cells are reachable by both arms ($\text{Reach}(L) = \text{Reach}(R) = 1$ for every cell), and all clearance margins lie between approximately 0.36 m and 0.52 m, well above the 0.15 m threshold. Thus, every board position is not only reachable but also sits deeply inside a safe region of each arm's workspace, indicating that the Tic-Tac-Toe task is not geometry-limited for either manipulator.

D. Test 2 – Workspace Redundancy and Partition Justification

Using the binary reachability from Test 1, we classify each cell as left-only, right-only, both, or none, and visualize the result as a 3×3 redundancy map. Fig. 5 shows that all nine cells fall into the “Both” category: every position on the board is safely reachable by both manipulators with margins above the safety threshold. There are no left-only, right-only, or unreachable cells.

This full redundancy means the board lies entirely within the overlapping workspace of the two arms. While this does provide maximum flexibility, it also increases the risk of arm–arm

interference if both planners operate over the same targets. These results justify treating coordination as a software problem and motivate our lateral workspace partition and MutexNode-based mutual exclusion for the shared central zone, as introduced in Section III-D.

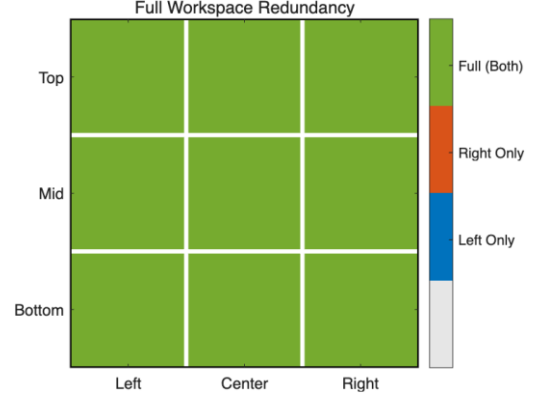


Figure 5 Redundancy map over the 3×3 board.

V. RESULTS AND DISCUSSION

A. Gazebo Simulation Results

We first validate the full Perception–Planning–Execution pipeline in Gazebo using the dual-arm Gen3 Lite models. The same RGB-D perception, board-state reconstruction, workspace-partition strategy, and MutexNode coordination described in Sections III and IV are used without modification. Figure 6 shows the simulated setup and a representative sequence of moves during an autonomous Tic-Tac-Toe game.

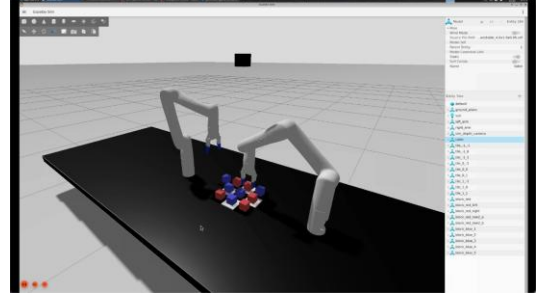


Figure 6 Gazebo dual-arm simulation executing the Tic-Tac-Toe task

Across multiple simulated games, both arms were able to grasp, transport, and place blocks onto the cells assigned to them by the partition strategy. We observed no self-collisions or planner deadlocks, and MoveIt consistently generated feasible trajectories for every scripted action. There were a few minor replanning events when an initial sampled path passed too close to the other arm, but the MutexNode and the lateral territory assignments ensured that the two manipulators never entered the shared center region at the same time. These results show that, given the kinematically safe workspace characterized in MATLAB, the high-level coordination strategy is enough to produce stable dual-arm behavior in dynamic simulation.

B. Real-World Experiments

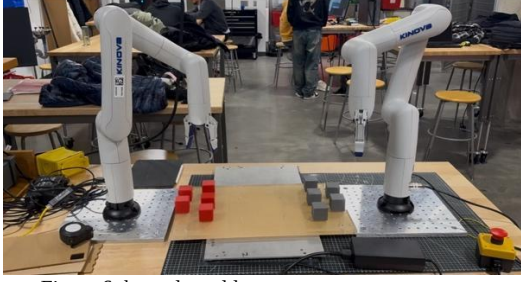


Figure 6 the real-world setup

We then deploy the same pipeline on the physical dual-Gen3 Lite setup. The only differences from simulation are the calibrated camera-to-world transform and the physical placement of the board and robots; the perception, planning, and execution code is reused directly, reflecting the sim-to-real design of our system.

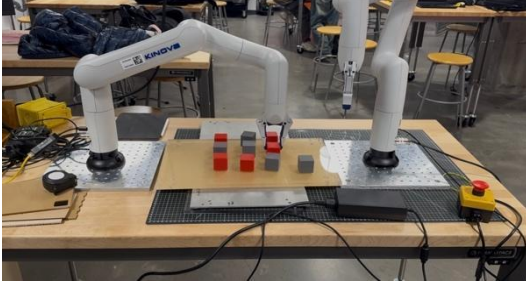


Figure 7 Test scene

Figure 7 illustrates the real-world setup, and Figure 8 shows an example RViz visualization where the point cloud, board model, and robot kinematics are consistently aligned.

In a series of real-world trials, each arm was able to pick up blocks from a staging area and place them onto its assigned board cells without human intervention. The RGB-D-based perception and YAML world-state interface were sufficient to reconstruct the evolving game state, and the action scripts generated by the planner translated into reliable joint-space motions on hardware. Importantly, we did not observe any arm-arm contact or near-collision events during these trials, which is consistent with the clearance margins and redundancy structure predicted by the MATLAB workspace analysis.

C. Overall Discussion

Overall, our results show a consistent story from kinematics to deployment. The MATLAB analysis confirms that all nine board cells are safely reachable by both arms with large clearance margins, so the Tic-Tac-Toe task is coordination-limited rather than geometry-limited. The workspace partition and MutexNode then turn this fully overlapping workspace into two predictable territories with a controlled shared zone. Gazebo simulations and real-world trials both validate this

design: using the same Perception-Planning-Execution pipeline, the dual-Gen3 Lite system executes games without arm-arm contact, and the observed behavior matches the redundancy and safety structure predicted by the workspace analysis.

VI. CONCLUSION

In this work, we presented a dual-arm robotic system that autonomously plays Tic-Tac-Toe using a modular Perception-Planning-Execution pipeline and a coordinated pair of Kinova Gen3 Lite manipulators. Rather than treating the task as a pure game-logic problem, we focused on the underlying workspace and coordination structure of the two arms. A MATLAB-based kinematic analysis showed that all nine board cells are safely reachable by both manipulators with generous clearance margins, implying that the task is fundamentally coordination-limited rather than geometry-limited. Motivated by this observation, we introduced a simple but effective workspace-partition strategy together with a MutexNode that enforces mutual exclusion in a small shared zone.

With this design, the same perception, planning, and execution code runs in both Gazebo and the real dual-Gen3 Lite setup without any changes. In both simulation and hardware tests, the system is able to play full games without the arms coming into contact. These results suggest that combining offline workspace analysis with simple, geometry-aware coordination rules is an effective way to achieve safe and predictable behavior in compact dual-arm workspaces. In the future, the same approach could be extended to more complex tasks, dynamic environments, or adaptive partitioning strategies beyond the Tic-Tac-Toe setting.

AI DISCLOSURE

AI-assistant tools were used only to enhance the clarity of writing; all engineering, analysis, and experimental components of the project were conducted by the team.

REFERENCES

- [1] Smith, C., Karayiannidis, Y., Nalpantidis, L., Gratal, X., Qi, P., Dimarogonas, D. V., & Kragic, D. (2012). Dual arm manipulation—A survey. *Robotics and*
- [2] Caccavale, F., & Uchiyama, M. (2008). Cooperative robot manipulation. *Springer Handbook of Robotics*.
- [3] Chitta, S., Sucan, I., & Cousins, S. (2012). MoveIt!: An open source robotics manipulation platform. *IEEE International Conference on Robotics and Automation (ICRA)*.
- [4] Vahrenkamp, N., Asfour, T., & Dillmann, R. (2012). Robot placement based on reachability inversion. *IEEE International Conference on Robotics and Automation (ICRA)*.
- [5] Åblad, E., Behrens, J. K., & Lennartson, B. (2023). Spatial–temporal load balancing and coordination of multi-robot systems. *Robotics and Computer-Integrated Manufacturing*, 83, 102606.
- [6] Behrens, J. K., Åkesson, K., & Lennartson, B. (2020). Simultaneous task allocation and motion scheduling for industrial robots. *Robotics and Computer-Integrated Manufacturing*, 65, 101962.
- [7] Sun, J., Li, Z., Li, H., & Xu, Q. (2025). Research on non-probabilistic reliability partitioning of robot workspace under parameter uncertainty. *Machines*, 13(1), 35.
- [8] Wessén, J., Åblad, E., & Stork, J. (2023). A constraint programming model for the scheduling and coordination of multi-arm robots using workspace zones. *Constraints*, 28(3), 325–360.